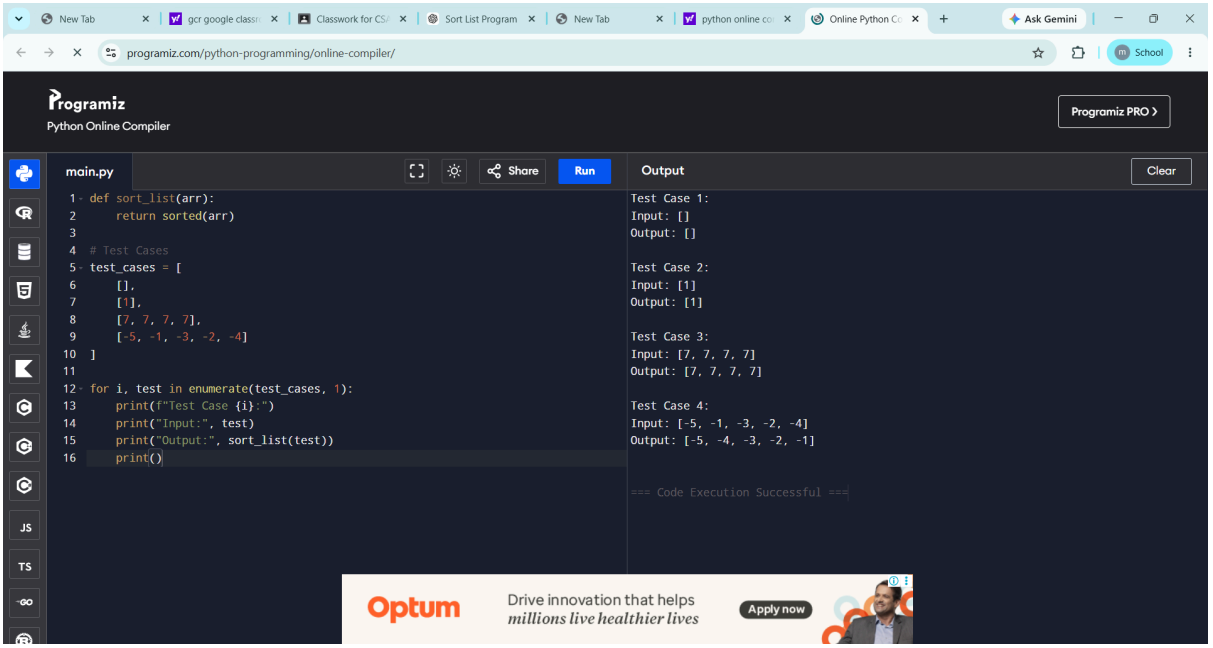
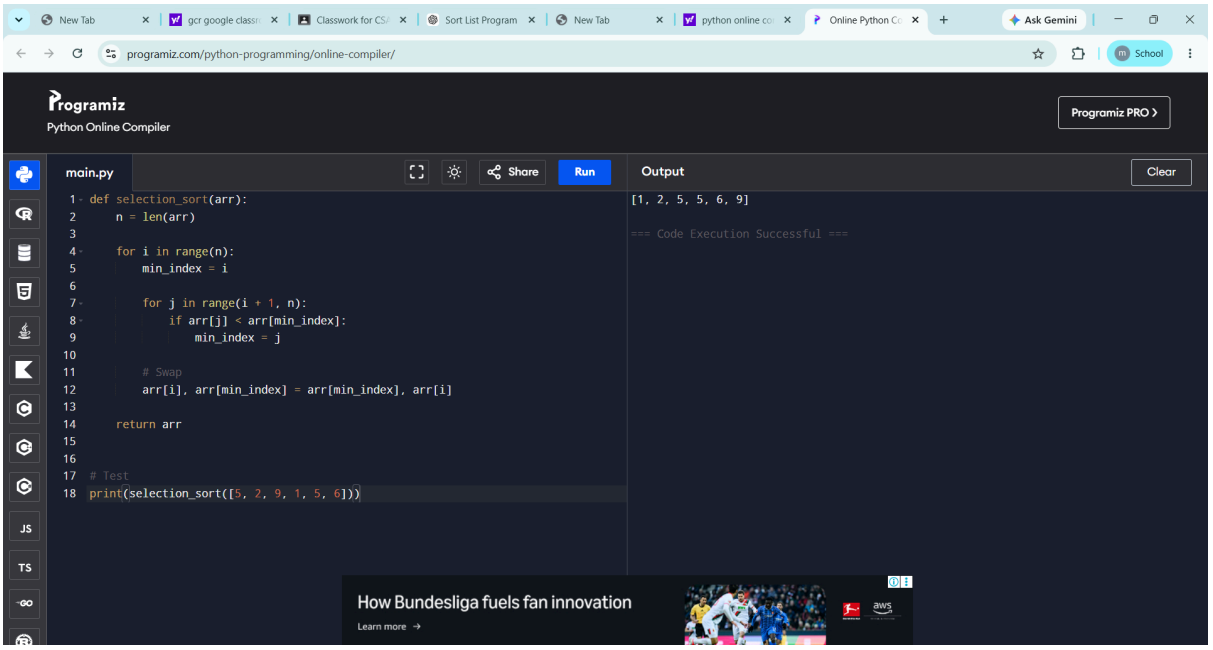
1. Write a program to perform the following
o An empty list
o A list with one element
o A list with all identical elements
o A list with negative numbers
Test Cases:
1. Input: []
o Expected Output: []
2. Input: [1]
o Expected Output: [1]
3. Input: [7, 7, 7, 7]
o Expected Output: [7, 7, 7, 7]
4. Input: [-5, -1, -3, -2, -4]
o Expected Output: [-5, -4, -3, -2, -1]

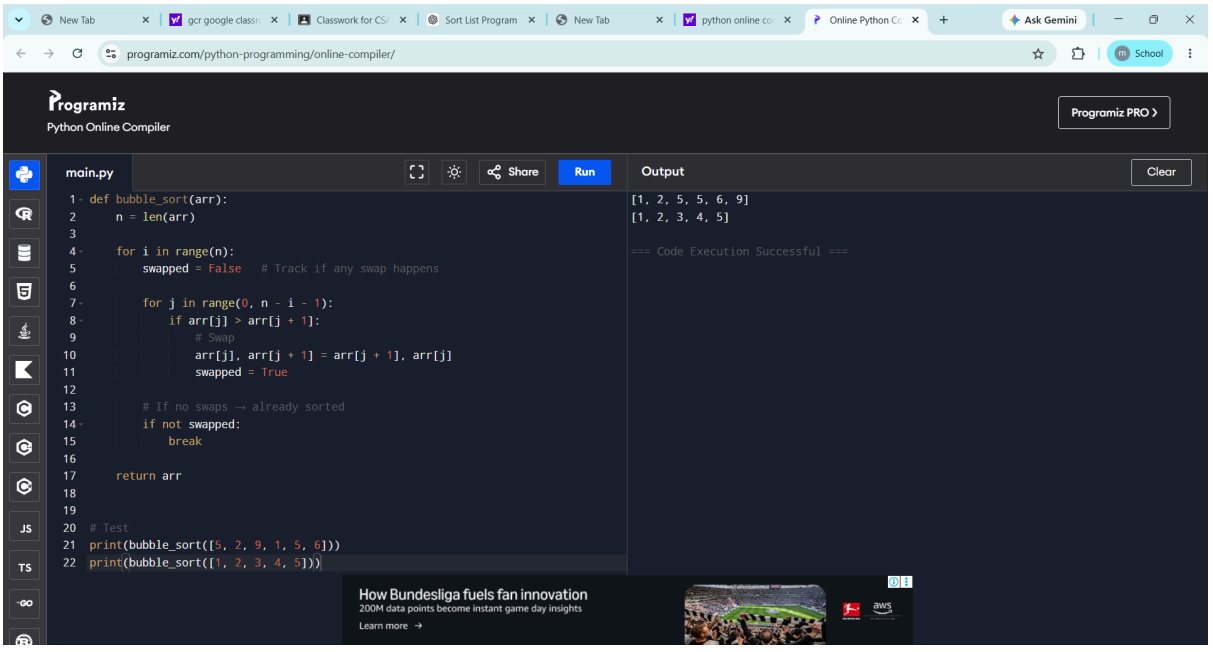


2. Describe the Selection Sort algorithm's process of sorting an array. Selection Sort works by dividing the array into a sorted and an unsorted region. Initially, the sorted region is empty, and the unsorted region contains all elements. The algorithm repeatedly selects the smallest element from the unsorted region and swaps it with the leftmost unsorted element, then moves the boundary of the sorted region one element to the right. Explain why Selection Sort is simple to understand and implement but is inefficient for large datasets. Provide examples to illustrate step-by-step how Selection Sort rearranges the elements into ascending order, ensuring clarity in your explanation of the algorithm's mechanics and effectiveness.
Sorting a Random Array:
Input: [5, 2, 9, 1, 5, 6]
Output: [1, 2, 5, 5, 6, 9]
Sorting a Reverse Sorted Array:
Input: [10, 8, 6, 4, 2]
Output: [2, 4, 6, 8, 10]
Sorting an Already Sorted Array:

Input: [1, 2, 3, 4, 5]
Output: [1, 2, 3, 4, 5]

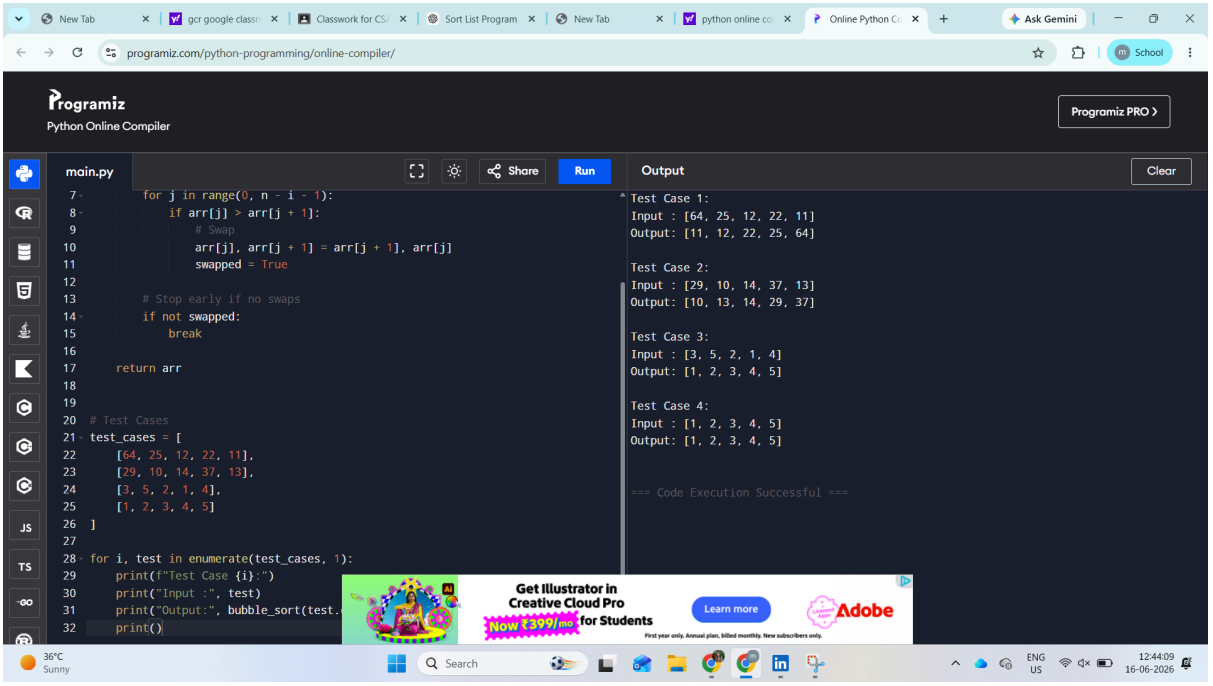


3. Write code to modify bubble_sort function to stop early if the list becomes sorted before all passes are completed.

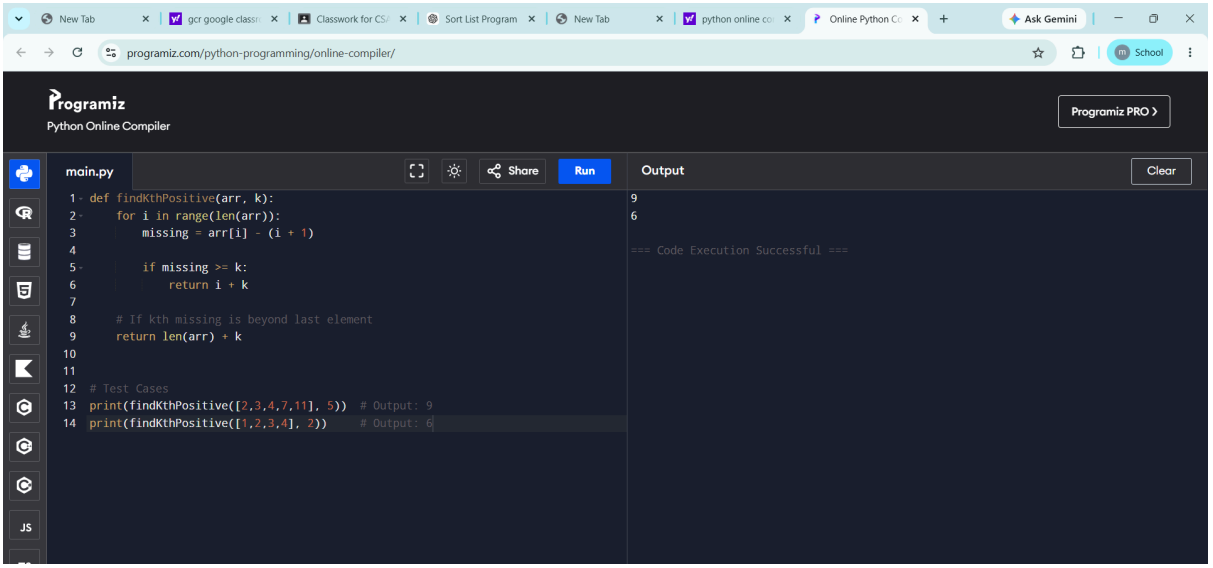


4. Test Cases:
Test your optimized function with the following lists:
1. Input: [64, 25, 12, 22, 11]
Expected Output: [11, 12, 22, 25, 64]
2. Input: [29, 10, 14, 37, 13]
Expected Output: [10, 13, 14, 29, 37]
3. Input: [3, 5, 2, 1, 4]

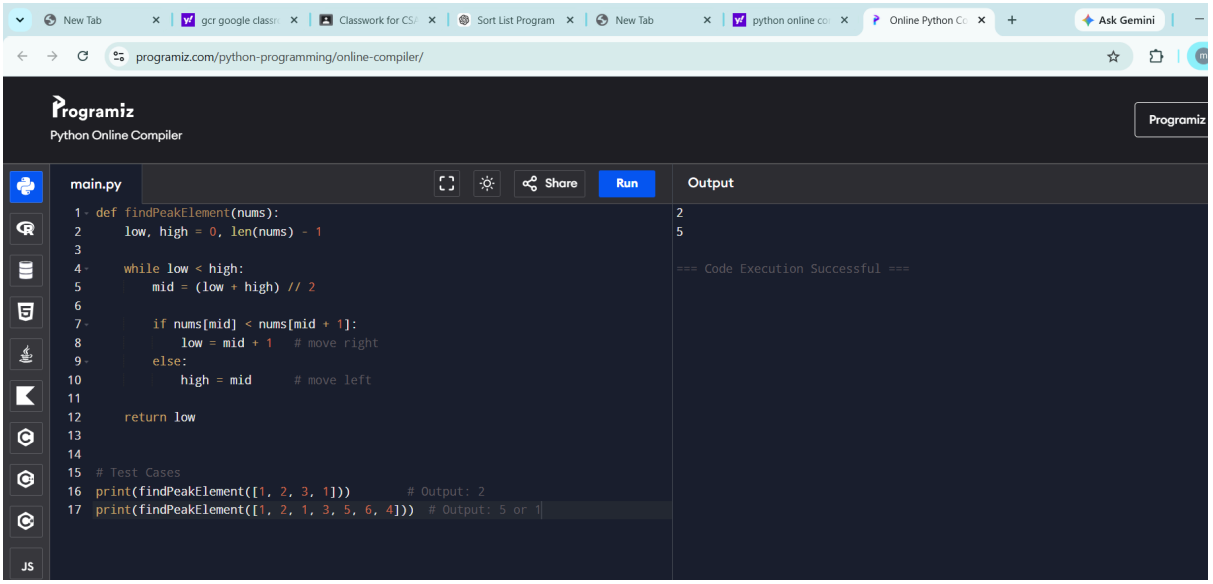
Expected Output: [1, 2, 3, 4, 5]
4. Input: [1, 2, 3, 4, 5] (Already sorted list)
Expected Output: [1, 2, 3, 4, 5]



5. Given an array arr of positive integers sorted in a strictly increasing order, and an integer k. return
the kth positive integer that is missing from this array.
Example 1:
Input: arr = [2,3,4,7,11], k = 5
Output: 9
Explanation: The missing positive integers are [1,5,6,8,9,10,12,13,...]. The 5th missing positive integer is 9.
Example 2:
Input: arr = [1,2,3,4], k = 2
Output: 6
Explanation: The missing positive integers are [5,6,7,...]. The 2nd missing positive integer is 6.



6. A peak element is an element that is strictly greater than its neighbors. Given a 0-indexed integer
array nums, find a peak element, and return its index. If the array contains multiple peaks, return
the index to any of the peaks. You may imagine that nums[-1] = nums[n] = -∞. In other words,
an element is always considered to be strictly greater than a neighbor that is outside the array.
You must write an algorithm that runs in O(log n) time.
Example 1:
Input: nums = [1,2,3,1]
Output: 2
Explanation: 3 is a peak element and your function should return the index number 2.
Example 2:
Input: nums = [1,2,1,3,5,6,4]
Output: 5
Explanation: Your function can return either index number 1 where the peak element is 2, or index number 5 where the peak element is 6.



7. Given two strings needle and haystack, return the index of the first occurrence of needle in
haystack, or -1 if needle is not part of haystack.
Example 1:
Input: haystack = "sadbutsad", needle = "sad"

Output: 0
Explanation: "sad" occurs at index 0 and 6.
The first occurrence is at index 0, so we return 0.
Example 2:
Input: haystack = "leetcode", needle = "leeto"
Output: -1
Explanation: "leeto" did not occur in "leetcode", so we return -1.